# DESIGN AND IMPLEMENTATION OF A SECURE FILE TRANSFER SYSTEM USING END-TO-END ENCRYPTION AND USER AUTHENTICATION

**A CAPSTONE PROJECT REPORT**

*Submitted in partial fulfilment of the requirements for the Course of*

**CAPSTONE PROJECT**

to the award of the degree of

**BACHELOR OF TECHNOLOGY**

**IN**

**INFORMATION TECHNOLOGY**

Submitted by

**Mukesh M (192521368)**

Under the Supervision of

**DR SALINI S**

## SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences**

Chennai – 602105

**SEPTEMBER 2026**

# SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences**

**Chennai-602105**

Engineer to Excel

# DECLARATION

We, MUKESH M of the Department of **B.TECH** [**Information Technology**], SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "**Design and Implementation of a Secure File Transfer System Using End-to-End Encryption and User Authentication**" is the result of our own bonafide efforts. To the best of our knowledge, the work presented in this report is original and has been carried out in accordance with the principles of engineering ethics and academic integrity.

Place:

Date:

**Signature of the Student with Names**

**MUKESH M (192521368)**

# SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences**

**Chennai-602105**

Engineer to Excel

# BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**Design and Implementation of a Secure File Transfer System Using End-to-End Encryption and User Authentication**" has been carried out by Mukesh M. under the supervision of Guide Name and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech [**Information Technology**] programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

**Course Co - ordinator:**

**Dr. S . SIVAGAMI Professor,**

**Department of Data Vista / CSE,**

**SIMATS Engineering**

**SIMATS CHENNAI.**

**Course Faculty:**

**Dr . S . SALINI**

**Department of CSE**

**SIMATS Engineering**

**SIMATS CHENNAI.**

# ABSTRACT

Secure file exchange is an important requirement in academic, business, research and organizational environments where files may contain confidential or valuable information. Conventional file-transfer approaches can expose data to risks such as unauthorized access, interception, weak authentication, accidental disclosure and insecure storage. The project "Design and Implementation of a Secure File Transfer System Using End-to-End Encryption and User Authentication" addresses these concerns by proposing a web-based secure file transfer system in which registered users authenticate themselves before accessing file-transfer functions and files are protected through an encryption workflow. The proposed system follows a layered security approach. User registration and login provide identity verification, while authorization ensures that a user can access only the resources permitted to that account. Before storage or transfer, the file is processed through encryption so that a protected representation is handled instead of ordinary plaintext. An authorized receiving user can retrieve and decrypt a file through the permitted workflow. The system also provides upload, download, sharing between registered users, sent and received file views, deletion, file details, transfer status and security-status information. The methodology consists of requirement analysis, architecture design, database design, implementation, integration and testing. Functional tests verify registration, login, upload, sharing, download and deletion. Security-oriented tests address unauthorized access, invalid credentials, access-control enforcement, protected storage and file integrity. Performance measurements may be used to evaluate transfer and cryptographic processing for different file sizes. The expected engineering outcome is a working prototype demonstrating how authentication, authorization, encryption, secure storage and controlled file sharing can be integrated into a single file-transfer workflow. The project emphasizes confidentiality, integrity, usability, reliability and maintainability rather than treating encryption as an isolated feature.



**Keywords**: End-to-End Encryption, User Authentication, Authorization, Secure File Transfer, Data Security, File Integrity

# TABLE OF CONTENTS


| |
|---|
| DECLARATION |
| BONAFIDE CERTIFICATE |
| INDIVIDUAL CONTRIBUTION STATEMENT |
| ABSTRACT |
| LIST OF FIGURES |
| LIST OF TABLES |
| LIST OF ABBREVIATIONS / SYMBOLS |
| CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM |
| 1.1 Background |
| 1.2 Need for the Project |
| 1.3 Problem Statement |
| 1.4 Project Aim |
| 1.5 Project Objectives |
| 1.6 Scope |
| 1.7 Expected Engineering Outcome |
| CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS |
| 2.1 Review Method |
| 2.2 Review of Existing Work |
| 2.3 Comparative Analysis |
| 2.4 Research / Engineering Gap |
| 2.5 Proposed Contribution |
| CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY |
| 3.1 Requirements |
| 3.2 Constraints & Applicable Standards |
| 3.3 Design Specifications |
| 3.4 Alternative Solutions & Evaluation |
| 3.5 Selected Design & Detailed Design |
| 3.6 Trade-off Analysis |

| |
|---|
| 3.7 Sustainability, Ethics, Safety, Risk & Security |
| CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT |
| 4.1 Development Approach & Resources |
| 4.2 Software Implementation & Modern Tools Used |
| 4.3 Test Plan & Verification |
| 4.4 Results |
| 4.5 Data Analysis & Engineering Judgment |
| 4.6 Comparison with Existing Solutions & Achievement of Objectives |
| 4.7 Strengths & Limitations |
| 4.8 Project Planning, Roles & Budget |
| CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION |
| 5.1 Conclusion |
| 5.2 Future Work |
| 5.3 Professional Learning & Individual Reflection |
| REFERENCES |
| APPENDICES |
| CAPSTONE PROJECT OUTCOME MAPPING SHEET |
| |


## LIST OF FIGURES


| Figure No. | Figure Name | Page |
|---|---|---|
| 3.1 | Overall system architecture | |
| 3.2 | Authentication workflow | ___ |
| 3.3 | Secure upload and encryption workflow | ___ |
| 3.4 | Sharing and download workflow | ___ |
| 4.1 | Application dashboard | ___ |
| 4.2 | Login and registration interface | ___ |
| 4.3 | File upload and sharing interface | ___ |

# CHAPTER 1 INTRODUCTION AND ENGINEERING PROBLEM

## 1.1 Background

Digital files are continuously exchanged between students, employees, researchers, departments and organizations. Documents, images, source code, reports and datasets may contain information that should not be exposed to unauthorized users. A file-transfer application therefore needs more than upload and download functionality. It should establish user identity, control access, protect file contents and provide a reliable method for authorized recipients to retrieve the correct file. The proposed secure file transfer system combines user authentication, authorization, encrypted file handling, secure storage, controlled sharing and file management. The system associates file operations with authenticated users and maintains relationships between users and files. The design also considers usability because security controls that are difficult to use may encourage unsafe workarounds.

## 1.2 Need for the Project

Basic file-sharing mechanisms may depend on passwords, shared links or storage locations without sufficient control over individual file access. If an account or storage location is compromised, unprotected files may be exposed. A secure file-transfer application reduces this exposure by combining identity verification, authorization and encryption. The engineering significance lies in integrating these mechanisms while maintaining acceptable usability and performance.

## 1.3 Problem Statement

The engineering problem is to design and implement a secure file transfer system that allows authenticated users to upload, store, share, download and delete files while reducing the risk of unauthorized access and exposure of file contents. The system must verify identity, enforce authorization, protect file contents through encryption, maintain correct user-file relationships and operate within realistic constraints of time, computing resources, storage and network performance.

## 1.4 Project Aim

The aim is to design and implement a secure, user-authenticated file transfer application that protects file contents through end-to-end encryption principles and provides controlled file sharing between authorized users.

## 1.5 Project Objectives

- Analyze security requirements associated with digital file transfer and storage.
- Design authentication and authorization for registered users.
- Design an encryption workflow that protects file contents.
- Implement secure upload, download, sharing, receiving and deletion.
- Maintain file metadata and user-file relationships in a database.
- Test functional correctness, access control, encryption workflow and integrity.
- Evaluate the system against objectives and identify limitations.

## 1.6 Scope

The project includes registration, login/logout, authenticated sessions, file upload, encryption, secure storage, download, authorized decryption, sharing between registered users, sent/received file views, file deletion, file details, transfer status and security-status information. It can be implemented as a web application with frontend, backend, database and file storage. Enterprise-scale deployment, hardware security modules, national-scale identity systems, formal certification and other advanced infrastructure are outside the basic scope and may be treated as future enhancements. The design assumes that established cryptographic libraries are used instead of custom cryptographic algorithms.

## 1.7 Expected Engineering Outcome

The expected outcome is a functional secure file-transfer prototype demonstrating the workflow from authentication to protected file exchange. The application should provide a clear dashboard and should reject unauthorized file operations. Final performance and security claims must be supported by actual test evidence.

# CHAPTER 2

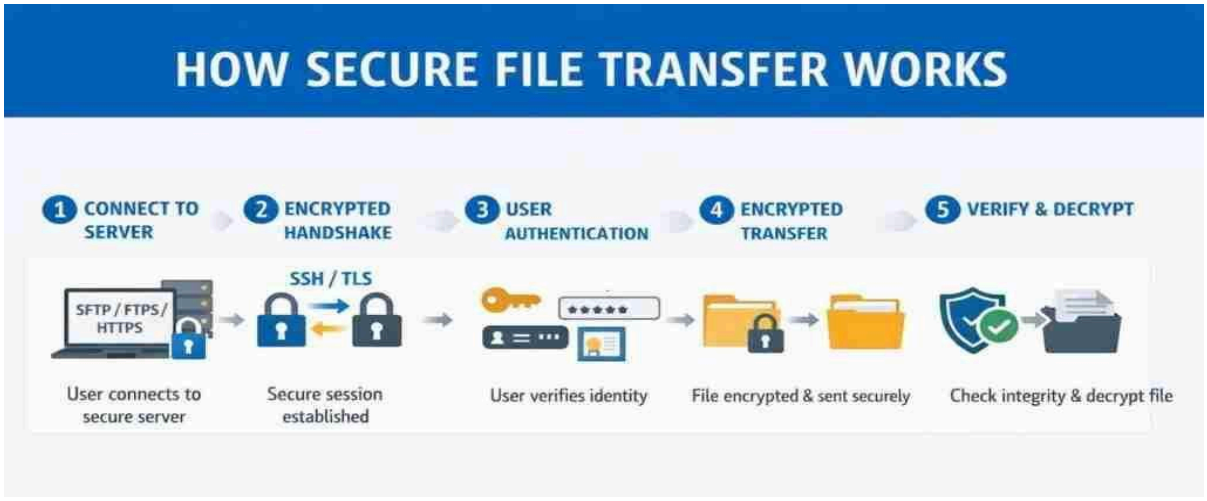
# LITERATURE REVIEW AND EXISTING SOLUTIONS

## 2.1 Review Method

The review is organized around authentication, authorization, encryption, secure communication, file integrity, secure storage and access control. Relevant research papers, standards, technical documentation and existing file-sharing approaches should be examined. The review should prioritize sources that explain security properties and engineering trade-offs.

## 2.2 Review of Existing Work

Traditional file transfer can be performed through direct network protocols or shared storage services. These approaches simplify file movement, but security depends on protocol configuration, authentication, storage protection and permissions. If file contents remain in plaintext at a storage location, compromise of that location can expose the information even when the network connection itself is protected. Modern secure applications commonly use encrypted network connections and authenticated sessions. Transport protection reduces interception risk while data moves across a network, whereas application-level encryption can add protection to stored files. Authentication confirms identity, while authorization determines which resources and operations an identity may use. Symmetric encryption is generally suitable for protecting large file contents because it is computationally efficient. Key management is therefore a critical engineering concern. Integrity protection is also important because a successful transfer should deliver the intended file without undetected modification. Cloud and enterprise file-sharing systems demonstrate the value of accounts, access permissions, sharing



## 2.3 Comparative Analysis

| Approach | Advantages | Limitations | Relevance |
|---|---|---|---|
| Basic file sharing | Simple | Weak access control; possible plaintext exposure | Motivates security controls |
| Traditional FTP-style transfer | Established model | Security depends on protocol/configuration | Background for secure transfer |
| Cloud file storage | Convenient and scalable | External trust and configuration dependence | Inspires sharing features |
| Proposed system | Integrated authentication, authorization and encryption | More implementation complexity | Directly addresses project objectives |

## 2.4 Research / Engineering Gap

The engineering gap is the integration of multiple security functions into one manageable workflow. Authentication alone does not protect stored files, while encryption without access control can create key-management and usability challenges. The project therefore integrates identity verification, authorization, encrypted file handling, controlled sharing and verification.

## 2.5 Proposed Contribution

The contribution is a practical application architecture for secure file transfer that can be implemented and tested in an academic environment. It does not claim to invent a new cryptographic algorithm; instead, it integrates established security principles into a coherent application and provides a usable dashboard for file operations.

# CHAPTER 3 - ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

## 3.1 Requirements

### 3.1.1 Stakeholder / User Requirements

Users require a simple mechanism to register, authenticate, upload files, share files with another registered user, receive files, download authorized files and delete files. They also require clear success and error messages. The system must prevent a user from accessing another user's private files unless a sharing relationship authorizes the access.

| Stakeholder | Requirement |
|---|---|
| Sender | Securely upload and share a file with an authorized recipient. |
| Receiver | View and download only authorized received files. |
| Maintainer | Operate the application, database and storage securely. |
| Assessor/Organization | Evidence of security, testing and engineering decisions. |

### 3.1.2 Functional and Non-Functional Requirements

| Requirement | Type | Target |
|---|---|---|
| Registration | Functional | Valid account created; duplicates rejected. |
| Login | Functional | Valid credentials accepted; invalid credentials denied. |
| Upload | Functional | Authenticated user can upload supported files. |
| Encryption | Security | Stored/transferred representation is protected. |
| Sharing | Functional | File can be shared with registered recipient. |
| Authorization | Security | Unauthorized file requests denied. |
| Integrity | Security | Retrieved file verified against original. |
| Performance | Non-functional | Acceptable response time in test environment. |
| Usability | Non-functional | Common tasks are clear and manageable. |

## 3.2 Constraints & Applicable Standards

Constraints include development time, available hardware, network conditions, storage capacity, software compatibility and key-management complexity. Established cryptographic libraries should be used instead of custom cryptographic algorithms. Security practices should align with recognized application-security, authentication and secure-communication guidance.

| Guidance/Standard | Application |
|---|---|
| TLS/HTTPS | Protect application communication in deployment. |
| OWASP security guidance | Address common web security and access-control risks. |
| Established cryptographic standards/libraries | Use reviewed cryptographic primitives and libraries. |
| IEEE-style referencing | Maintain consistent technical references. |

## 3.3 Design Specifications

| Parameter | Target | Priority |
|---|---|---|
| Authentication | Unique account with protected password verification | High |
| Authorization | Only permitted files/operations accessible | High |
| Confidentiality | File contents protected using encryption | High |
| Integrity | Detect unintended modification | High |
| Availability | Usable for intended test workload | Medium |
| Usability | Clear dashboard and status feedback | Medium |
| Maintainability | Modular application design | Medium |

## 3.4 Alternative Solutions & Evaluation

Alternative 1 is a simple login-based file application with ordinary server storage. It is easy to implement but provides weaker protection if stored files are exposed. Alternative 2 relies mainly on encrypted network transport, which protects communication but does not fully protect files stored on the server. Alternative 3 combines authentication, authorization, encrypted file handling, controlled sharing and integrity verification, providing stronger layered protection at the cost of additional complexity.

| Criterion | Weight | Alt. 1 | Alt. 2 | Alt. 3 |
|---|---|---|---|---|
| Security | 30% | Low | Medium | High |
| Complexity | 20% | Low | Medium | High |
| Performance | 15% | High | High | Medium |
| Usability | 15% | High | High | High |
| Maintainability | 10% | Medium | Medium | High |
| Scalability | 10% | Low | Medium | Medium |

### 3.5 Selected Design & Detailed Design

Alternative 3 is selected because the main requirement is secure file exchange. The architecture separates the user interface, authentication and authorization, application logic, database, encryption/decryption service and protected file storage. The logical flow is User Interface → Authentication → Authorization → File Service → Encryption/Decryption → Storage and Database. During upload, the authenticated user selects a file and the backend validates the session, metadata and file constraints. The file is processed by an established encryption library and the protected representation is stored. Metadata records the owner, safe display name, size, time, storage reference and sharing information. Secret keys must never be exposed through the user interface. During sharing, the sender selects a registered recipient and the system creates an authorized sharing relationship. During download, the backend authenticates the request and verifies that the current user is permitted to retrieve the file. Only after authorization succeeds is the protected file processed for retrieval/decryption. Integrity information can be checked before the file is delivered.

Insert Figure 3.1: Overall Architecture; Figure 3.2: Authentication Workflow; Figure 3.3: Upload/Encryption Workflow; Figure 3.4: Sharing/Download Workflow.

### 3.6 Trade-off Analysis

| Decision | Alternative | Benefit | Disadvantage | Final Decision |
|---|---|---|---|---|
| Encryption | Plain / transport-only / application encryption | More protection for stored contents | Processing and key management | Use established application-level encryption. |
| Authentication | Password only / stronger authentication | Simple deployment | Password-only risk | Secure password storage; MFA as future work. |
| Storage | Plain / protected | Protected storage reduces exposure | Processing overhead | Store protected file representation. |
| Sharing | Public link / account-based | Account-based limits recipients | Requires recipient identity | Use registered-user sharing. |

### 3.7 Sustainability, Ethics, Safety, Risk & Security

| Domain | Evidence/Consideration | Impact |
|---|---|---|
| Sustainability | Storage duplication and file lifecycle | Deletion and file-management controls. |
| Ethics | Confidentiality and unauthorized disclosure | Restrict access to authorized accounts. |
| Safety | Software-based project; normal workstation safety | Follow safe workstation/electrical practice. |
| Security | Confidentiality, integrity, authentication and authorization | Layered security architecture. |

| Risk | Likelihood | Severity | Mitigation | Residual Risk |
|---|---|---|---|---|
| Weak password | Medium | High | Password policy and secure hashing | Medium |
| Unauthorized file access | Medium | High | Authorization on every protected operation | Low/Medium |
| Key exposure | Low/Medium | High | Protect keys; no hard-coded secrets | Medium |
| Malicious upload | Medium | Medium | Validate size/type and storage controls | Low/Medium |
| Database compromise | Low/Medium | High | Restrict DB access; protect sensitive values | Medium |
| Data loss | Low | High | Backup and recovery procedures | Low/Medium |

# CHAPTER 4 - IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

## 4.1 Development Approach & Resources

The project follows an incremental development approach. Requirements are converted into modules, followed by architecture and database design. Authentication is implemented before file operations because ownership and authorization depend on user identity. Encryption and storage are then integrated, followed by sharing, downloading, deletion and dashboard functions. Testing is performed throughout development so that errors are identified before final integration.

| Resource | Purpose |
|---|---|
| Development computer | Development and local testing |
| IDE/Code editor | Coding and debugging |
| Web browser | Frontend and workflow testing |
| Backend runtime/framework | Application logic and APIs |
| Database | Users, files, sharing and transfer metadata |
| File storage | Protected file objects |
| Cryptographic library | Encryption/decryption |
| Version control | Source-code management |

## 4.2 Software Implementation & Modern Tools Used

The frontend provides registration, login, dashboard, upload, file lists, sharing, received/sent views, file details and account controls. The backend provides authentication, authorization, file management, sharing and download services. Every protected file operation should verify the authenticated session and the relationship between the current user and requested resource.

The database stores account information and file metadata rather than relying only on filenames. A typical file record contains an internal identifier, owner identifier, safe display name, storage reference, file size, creation time, encryption status, integrity metadata and sharing status. Passwords are stored using an appropriate password-hashing mechanism and never as plaintext.

| Tool/Software | Purpose | Stage |
|---|---|---|
| HTML/CSS/JavaScript or frontend framework | User interface | Frontend |
| Backend framework/runtime | APIs and file operations | Backend |
| SQL DBMS | Persistent metadata | Database |
| Cryptographic library | Encryption/decryption | Security |
| Git/version control | Version tracking | Development |
| Browser developer tools | Debugging | Testing |

Insert Figure 4.1: Dashboard; Figure 4.2: Login/Registration; Figure 4.3: Upload/Sharing Interface.

### 4.3 Test Plan & Verification

Testing is organized around the requirements defined in Chapter 3. Functional testing verifies valid operations. Negative testing verifies rejection of invalid credentials, unauthorized file identifiers and invalid inputs. Security verification examines authorization and encrypted storage. Integrity testing compares the original and retrieved file using an appropriate verification method.

| Requirement | Test Method | Acceptance Criterion |
|---|---|---|
| Registration | Valid and duplicate registrations | Valid account created; duplicates rejected |
| Login | Valid and invalid credentials | Valid user accepted; invalid user denied |
| Upload | Upload supported file after login | File accepted and metadata created |
| Encryption | Inspect protected representation/status | File not stored as ordinary plaintext |
| Sharing | Registered/unregistered recipient | Only valid recipient assigned |
| Authorization | Access another user's file | Request rejected |
| Download | Authorized recipient download | Correct file retrieved |
| Integrity | Compare original/retrieved | Verification confirms unchanged file |
| Delete | Authorized deletion | File/metadata handled per policy |

### 4.4 Results

The final report must contain measurements obtained from the actual implemented system. The following table is a recording template and must be replaced with real evidence after execution. Test values must not be invented.

| Test Case | Required Value | Achieved Value | Status |
|---|---|---|---|
| TC01 Registration | Registration succeeds | [Enter result] | [Pass/Fail] |
| TC02 Login | Valid login succeeds | [Enter result] | [Pass/Fail] |
| TC03 Invalid login | Invalid login rejected | [Enter result] | [Pass/Fail] |
| TC04 Upload | Authorized upload succeeds | [Enter result] | [Pass/Fail] |
| TC05 Encryption | Protected storage confirmed | [Enter evidence] | [Pass/Fail] |
| | | | |

| | | | |
|---|---|---|---|
| TC06 Unauthorized access | Access denied | [Enter evidence] | [Pass/Fail] |
| TC07 Sharing | Recipient receives file | [Enter result] | [Pass/Fail] |
| TC08 Download/decryption | Correct file recovered | [Enter result] | [Pass/Fail] |
| TC09 Integrity | Original equals retrieved | [Enter hash/result] | [Pass/Fail] |
| TC10 Delete | Authorized deletion succeeds | [Enter result] | [Pass/Fail] |

### 4.5 Data Analysis & Engineering Judgment

Evaluation should consider both functionality and security. A system is not successful merely because files can be transferred; it must also reject unauthorized requests and preserve intended file contents. If encryption increases processing time, upload and download measurements should be compared with the security benefit. Performance tests can use several file sizes and record upload time, download time, encryption time and decryption time. If a test fails, the failure should document the probable cause, corrective action and retest result. This provides engineering evidence rather than unsupported claims.

### 4.6 Comparison with Existing Solutions & Achievement of Objectives

| Objective | Evidence | Achievement |
|---|---|---|
| Secure architecture | Architecture and workflow diagrams | [Fully/Partially/Not Achieved] |
| Authentication | Registration/login tests | [Fully/Partially/Not Achieved] |
| Encryption | Protected-storage evidence | [Fully/Partially/Not Achieved] |
| Sharing | Sent/received test | [Fully/Partially/Not Achieved] |
| Access control | Unauthorized-access test | [Fully/Partially/Not Achieved] |
| Integrity | Hash/comparison evidence | [Fully/Partially/Not Achieved] |

### 4.7 Strengths & Limitations

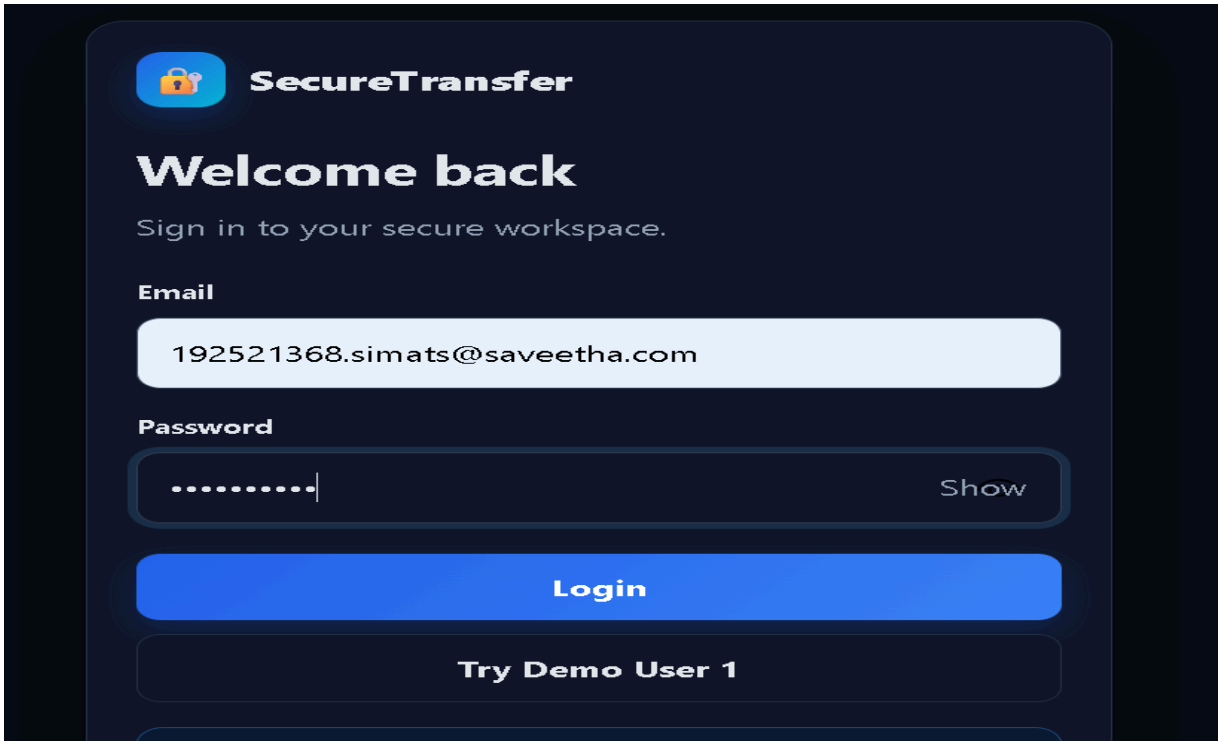
Strengths: The system integrates authentication, authorization, encryption, file management and controlled sharing. The modular architecture supports maintenance and testing. The dashboard can make file status and transfer operations understandable. The project provides a practical demonstration of application security principles.

Limitations: The prototype may be limited by development hardware and network resources. Key management may be simplified compared with enterprise systems. Multi-factor authentication, hardware-backed keys, large-scale distributed storage, advanced malware scanning, formal penetration testing and high-availability deployment may remain outside scope.

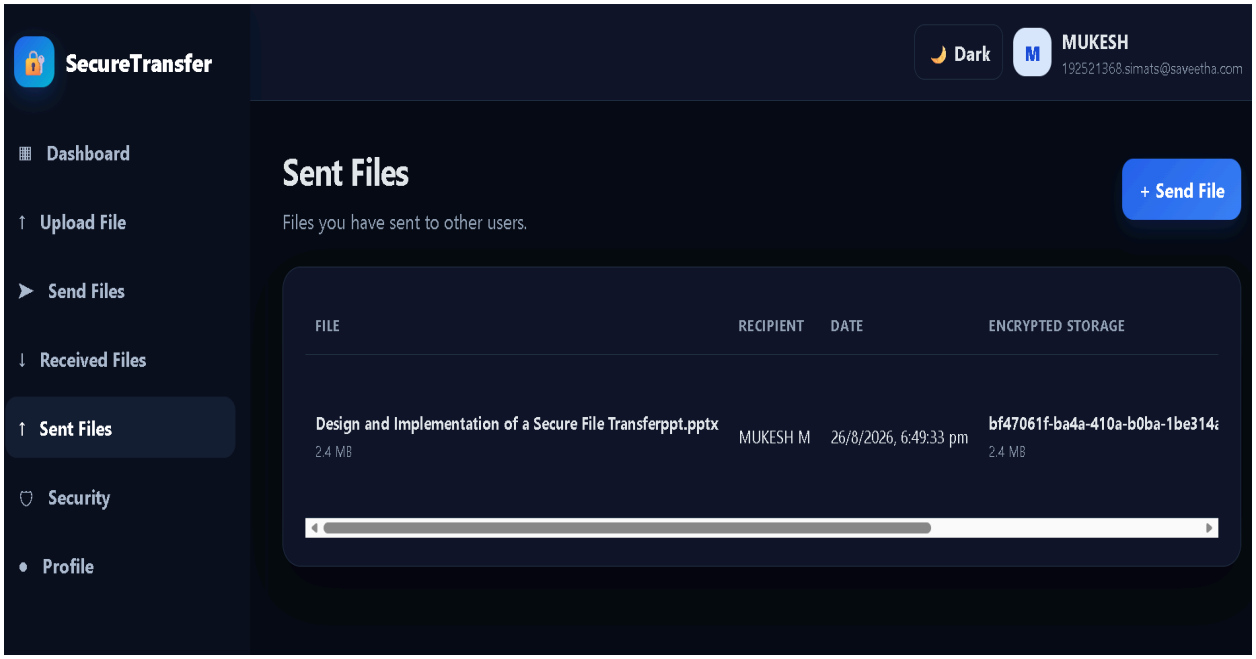
### 4.8 Project Planning, Roles & Budget

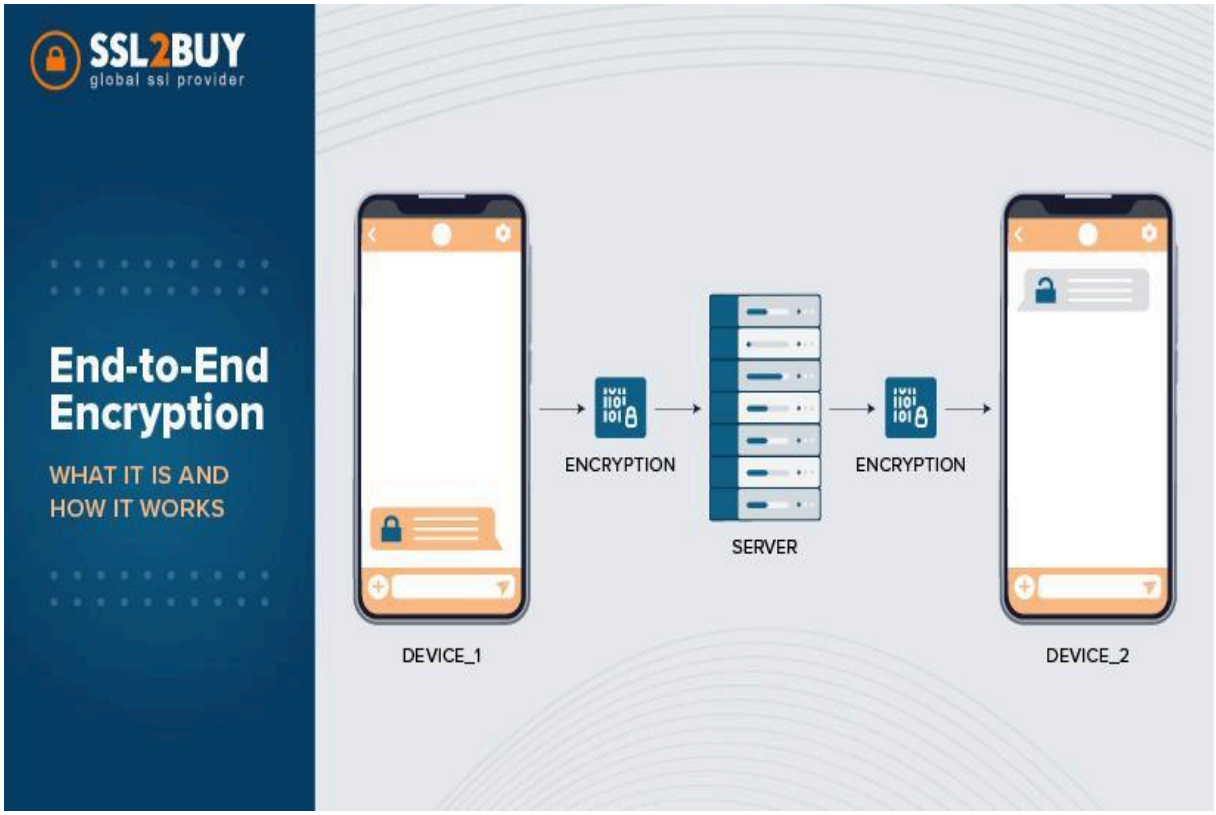
| Phase | Activity | Output |
|---|---|---|
| Week 1 | Requirements and problem definition | Requirements |
| Week 2 | Literature and architecture | Design |
| Week 3 | Database and authentication | User module |
| Week 4 | Encryption and storage | Secure file module |
| Week 5 | Sharing and download | Transfer module |
| Week 6 | Frontend/backend integration | Integrated app |
| Week 7 | Testing and debugging | Test evidence |
| Week 8 | Documentation and review | Final report |
| Item | Estimated Cost | Actual Cost |
| Development software | ₹0–₹2,000 | [Enter] |
| Hosting/deployment | ₹0–₹2,000 | [Enter] |
| Internet/data | [Enter] | [Enter] |
| Other expenses | [Enter] | [Enter] |
| Total | [Enter] | [Enter] |

## MODULE 1 User Authentication and Access Control Module




## MODULE 2 - Secure File Encryption and Transfer Module




SSL2BUY
global ssl provider
End-to-End Encryption
WHAT IT IS AND HOW IT WORKS
ENCRYPTION
SERVER
ENCRYPTION
DEVICE_1
DEVICE_2

# CHAPTER 5

# CONCLUSION, FUTURE WORK AND REFLECTION

## 5.1 Conclusion

The project addresses the engineering problem of exchanging digital files while reducing unauthorized access and data-exposure risks. The proposed solution combines authenticated accounts, authorization, encrypted file handling, controlled sharing, secure storage and file management into a unified workflow. The design uses layered security: authentication establishes identity, authorization controls protected resources, encryption protects file contents and integrity verification helps confirm that the retrieved file is the intended file. The design also considers usability, maintainability, performance and realistic development constraints. Final claims must be supported by actual Chapter 4 test results. Overall, secure file transfer is a system-level engineering problem involving user management, application logic, storage, cryptography, database design and testing. The prototype provides a foundation for stronger authentication, advanced key management, audit logging, scalable storage and production deployment.

## 5.2 Future Work

- Introduce multi-factor authentication.
- Improve cryptographic key management and protected key storage.
- Add security audit logs.
- Add malware/file-content scanning.
- Support resumable large-file transfers.
- Add fine-grained role and sharing permissions.
- Use hardened HTTPS deployment and secure secrets management.
- Perform formal security assessment and penetration testing.
- Add backup, recovery, retention and secure-destruction policies.
- Evaluate concurrent-user scalability.

## 5.3 Professional Learning & Individual Reflection

**New Knowledge Acquired**

- Authentication, authorization and encryption integration.
- Secure web application architecture.
- Database design for users, files and sharing.

- Security and functional testing.
- Importance of secure password and key handling.

**Engineering & Professional Skills Developed**

- Requirements analysis and system design.
- Frontend/backend integration.
- Database and API development.
- Security-focused problem solving.
- Testing, debugging and technical documentation.
- Project planning and communication.

**Individual Reflection**

The most difficult engineering challenge was integrating security controls without making ordinary file-transfer operations unnecessarily complicated. Designing the relationship between authenticated users, file ownership, recipients and authorization checks required careful consideration because access-control errors can expose another user's file. A key engineering decision was to use established cryptographic mechanisms and libraries instead of creating a custom encryption algorithm. This reduces unnecessary cryptographic risk and allows the project to focus on correct integration, key protection, authorization and verification. The project provided learning opportunities in secure application design, database integration, encryption workflows, authentication and testing. If repeated, greater attention would be given earlier to key management, threat modeling, automated security testing and deployment hardening.

## REFERENCES

[1] OWASP Foundation, OWASP Top 10: The Ten Most Critical Web Application Security Risks.

[2] OWASP Foundation, Application Security Verification Standard (ASVS).

[3] National Institute of Standards and Technology, Advanced Encryption Standard (AES), FIPS PUB 197.

[4] National Institute of Standards and Technology, Digital Identity Guidelines: Authentication and Lifecycle Management, NIST SP 800-63B.

[5] Internet Engineering Task Force, The Transport Layer Security (TLS) Protocol, RFC 8446.

[6] W. Stallings, Cryptography and Network Security: Principles and Practice, Pearson.

[7] W. Mao, Modern Cryptography: Theory and Practice, Prentice Hall.

[8] A. Menezes, P. van Oorschot and S. Vanstone, Handbook of Applied Cryptography, CRC Press.

[9] Peer-reviewed research papers on secure file sharing, end-to-end encryption, authentication and access control used in the final literature review.

[10] Documentation for the frameworks, database, cryptographic library and development tools actually used.

# APPENDICES

## Appendix A – Detailed Calculations

Include performance, throughput, file-size, encryption/decryption timing and other relevant calculations.

## Appendix B – Engineering Drawings / Schematics

Include system architecture, database relationship, authentication, encryption and deployment diagrams.

## Appendix C – Source Code / Repository Information

Include approved repository information and essential code excerpts only. Never include passwords, keys or credentials.

## Appendix D – Datasheets

Include relevant software or infrastructure specifications where required.

## Appendix E – Experimental / Raw Data

Include raw upload/download timings, file sizes, authentication results and integrity verification.

## Appendix F – Ethics / Institutional Approval

Include applicable approval documentation, if required.

## Appendix G – Project Meeting / Progress Records

Include supervisor meetings, review comments and milestone evidence.

## Appendix H – User / Industry Feedback

Include feedback, if applicable.

## Appendix I – Publications / Patents / Competitions / Product Outcomes

Include relevant outcome evidence, if applicable.

## Appendix J – Individual Contribution Evidence

Include commits, task records, screenshots or other contribution evidence.